

# SAFEGUARD

Women Safety & Emergency Assistance Platform

## PRODUCT REQUIREMENT DOCUMENT (PRD)

### Document Properties

**Document Version:** 1.0.0 (Production-Ready)  
**Release Date:** July 11, 2026  
**Target Role:** Full-Stack Product & Software Engineering Intern  
**Primary Author:** Aditya (Platform Creator)

Built with Node.js, Next.js, Express, Socket.io, and PostgreSQL/PostGIS

## 1. Executive Summary & Product Vision

Safeguard is a production-ready, ultra-responsive security hub designed to eliminate response delays during emergency distress events. The platform facilitates instant distress signal broadcasting via WebSockets, connects distressed citizens with emergency services, and runs proximity-based matchmaking algorithms to dispatch nearby community volunteers.

A key innovation of Safeguard is its flexible database adapter layer. By default, the system target environment couples with PostgreSQL and spatial indexing (PostGIS) to execute precise distance-based volunteer routing. To ensure zero-friction demonstrations, testing, and evaluation, the backend features an automated fallback engine. If the Postgres database connection is not active or set in environment variables, the engine boots using an in-memory db model pre-seeded with mockup parameters, proving production viability with zero environment setup overhead.

## 2. Problem Statement & Value Proposition

### The Critical Problem

- **Emergency Contact Gaps:** Traditional public response options (e.g. calling emergency services) have dispatch latencies, and verbal communications during distress are often difficult or impossible.
- **Underutilized Local Networks:** In high-density environments, bystanders or safety volunteers are often just hundreds of meters away but remain unaware of nearby distress incidents.
- **Information Disconnection:** Distressed users lack real-time confirmation showing whether assistance is on the way, causing severe panic, while family contacts are left in the dark.

### Value Proposition

- **One-Click Broadcast:** SOS triggers socket messages instantly, establishing connection logs and generating coordinates without requiring complex navigation.
- **Proximity Matchmaking:** Queries the database dynamically using 5km spatial queries to filter only online, approved, and verified volunteer helpers.
- **Bidirectional Location Tracking:** Aggregates coordinates dynamically, plotting responders and victims on a Leaflet map view with simulation mechanics.

### 3. User Personas & Role Requirements

The Safeguard safety system is customized to manage three primary personas. Each role accesses a specialized panel configured with unique tools and authorization scopes:

#### Persona 1: Distress User (Woman)

- **Context:** A citizen walking alone in unfamiliar areas or late at night who needs immediate safety support.
- **Key Goals:** Send an SOS warning immediately, notify emergency contacts, and see tracking paths of responders.
- **Functional Needs:** Fast dashboard panel containing a cancellable countdown warning button, safety contacts registration form, and a map view showing nearby police, hospitals, and responder movements.

#### Persona 2: Verified Volunteer / Responder

- **Context:** A community-minded citizen who wants to act as a first-line safety helper in their local area.
- **Key Goals:** Set availability status, receive active local alerts, and navigate safely to the incident.
- **Functional Needs:** Availability status toggle (Online/Offline), incoming distress cards showing distance, simulated step navigator to report mock location updates, and post-resolution report logs.

#### Persona 3: System Administrator

- **Context:** A municipal operator or emergency services manager supervising safety systems.
- **Key Goals:** Oversee active alerts, verify volunteer qualifications, and manage safety coordinates catalog.
- **Functional Needs:** Central KPI board (alert success ratios, monthly active users, response speeds), Recharts weekly trends graphical display, pending volunteer verification approval table, and CRUD resource catalog forms.

## 4. Functional Requirements & Features

### 4.1 One-Touch SOS Distress Button

The central utility of the distress user console. It initiates a 3-second red pulsing countdown, offering a cancel option to prevent accidental triggers. Upon countdown completion, the system logs the incident as "Active" in the database, broadcasts the emergency to WebSockets, and logs simulated SMS messages to family contacts with live tracking URLs.

### 4.2 Real-time Interactive Tracking Map

A Zero-API-key mapping dashboard built with Leaflet.js and OpenStreetMap. When an alert is accepted, it displays coordinates of the user and responding volunteer. It overlays nearby safety points (Police Stations, Safe Zones, Hospitals) within the area.

### 4.3 Volunteer / Responder Console

Allows registered volunteers to manage their availability. When a proximity alert occurs, the volunteer console triggers dynamic notifications. If accepted, the status updates to "Accepted" globally. The console provides a "Simulate Step Closer" button, letting the volunteer post simulated coordinate steps, which updates the victim's screen live.

### 4.4 Centralized Admin Dashboard & KPI Monitor

Aggregates statistics showing critical safety metrics:

- **Success Rate:** Calculated dynamically on the server as:  $\text{Resolved} / (\text{Resolved} + \text{Cancelled})$ .
- **Average Response Time:** Logs average elapsed time between SOS trigger and volunteer acceptance.
- **Directory Approval Panels:** Allows administrators to toggle volunteer statuses from Pending to Approved or Rejected.
- **Resource Management Form:** Central catalog panel to insert, modify, and delete safe havens, hospitals, and police locations.

## 5. System Architecture & Real-Time Data Flow

### 5.1 Architecture Stack & System Flow

The platform uses a modern decoupled stack: Next.js App Router for client screens, Node.js and Express for backend REST APIs, Socket.io for bidirectional communication, and PostgreSQL/PostGIS for geographic queries, with an in-memory JavaScript database fallback engine.

### 5.2 Real-Time WebSocket Messaging Protocol

| Event Name                | Sender -> Recipient   | Purpose / Description                                         |
|---------------------------|-----------------------|---------------------------------------------------------------|
| join-alert-room           | Client -> Server      | Subscribes client socket to unique room: alert_{alertId}      |
| leave-alert-room          | Client -> Server      | Unsubscribes client from tracking room                        |
| new-sos-alert             | Server -> Volunteer   | Broadcasts active SOS coordinates warning to volunteers       |
| alert-updated             | Server -> All Clients | Notifies changes in incident statuses (Accepted, Resolved)    |
| volunteer-location-share  | Volunteer -> Server   | Transmits volunteer movement step coordinate data             |
| volunteer-location-update | Server -> Room        | Relays volunteer movement updates to the victim tracking page |

### 5.3 Geospatial Proximity Matchmaking Algorithm

Production Spatial Search Query (PostGIS):

```
SELECT * FROM "Volunteers" WHERE "verificationStatus" = 'Approved' AND "isOnline" = true AND
ST_DWithin("currentLocation", ST_SetSRID(ST_MakePoint(long, lat), 4326)::geography, 5000);
```

Fallback Haversine Formula Algorithm (In-Memory Fallback):

```
const a = Math.sin(dLat/2)**2 + Math.cos(lat1)*Math.cos(lat2)*Math.sin(dLon/2)**2; const c =
2 * Math.atan2(Math.sqrt(a), Math.sqrt(1-a)); const distance = R * c;
```

## 6. Database Schema & Data Models

The PostgreSQL / Sequelize database model structures are defined below, maintaining type-safety and foreign key constraints:

### User Model (Identity & Auth)

- **\_id:** **UUID** (Primary Key, Autogenerated)
- **name / email:** **String** (Unique, Not Null credentials)
- **passwordHash:** **String** (Bcrypt encrypted password hash)
- **role:** **Enum** ("User", "Volunteer", "Admin")
- **phone:** **String** (Not Null Mobile phone contact details)

### Volunteer Model (Responders Registry)

- **\_id / userId:** **UUID** (Unique Identifier linking to Core User model)
- **verificationStatus:** **Enum** ("Pending", "Approved", "Rejected")
- **isOnline:** **Boolean** (Default: false availability switch)
- **currentLocation:** **Point** (PostGIS Spatial Geography Coordinate)

### Alert Model (Active Emergency Incidents)

- **\_id / userId:** **UUID** (Foreign Keys representing the distress victim)
- **status:** **Enum** ("Active", "Accepted", "Resolved", "Cancelled")
- **location:** **Point** (Geospatial Coordinates of the SOS signal)
- **responderId:** **UUID** (Foreign Key linking to active Volunteer)

### EmergencyContact & SafetyResource Models

- **EmergencyContact:** **Schema** (\_id, userId, name, phone, relationship, isNotifiedBySOS)
- **SafetyResource:** **Schema** (\_id, name, type ["SafeZone", "PoliceStation", "Hospital"], location, address, phone)

### IncidentReport Model (Post-Event Logs)

- **IncidentReport:** **Schema** (\_id, alertId, userId, volunteerId, notes, severity ["Low", "Medium", "High"])

## 7. Non-Functional Requirements & Security

### Performance & Latency

- **Real-time Sync Latency:** WebSocket packet transfers must maintain a sub-500ms roundtrip to prevent visual lagging on tracking maps.
- **Geospatial Query Speeds:** Radius computations must execute in under 100ms on database instances.

### Security & Encryption

- **Hashing Safeguard:** Passwords must be securely encrypted via bcrypt hashing algorithm (salt factor 10) before storage.
- **Access Tokens:** API routes are guarded by JWT tokens, verifying credentials and scopes on request entries.

### Privacy Boundaries

- **Location Access Bounds:** A user's live coordinates are strictly restricted. Location streaming is authorized and accessible to volunteers only during active distress windows.

## 8. Future Enhancements & Product Roadmap

- **Wearable Integration:** Companion mobile software synced with safety rings, wristbands, or smartwatches to trigger SOS signals without opening the phone console.
- **Native SMS Gateways:** Integration with Twilio or regional gateways to transmit locations via fallback text messages during data network losses.
- **Geofencing Warnings:** Alert warning functions notification flags when users steer into historically unsafe boundaries.